Journal of Machine Learning Research XX (2026) 1–8 Submitted MM/YY; Published MM/YY

fastEmbedR: Native CPU, Metal, and CUDA Implementations of UMAP and openTSNE for R

Moussa Kassim[1,2] Moussa.Kassim@icgeb.org
Martin Ocharo[1,2] Martin.Ocharo@icgeb.org
Alessia Vignoli[3,4] vignoli@cerm.unifi.it
Leonardo Tenori[3,4] tenori@cerm.unifi.it
Stefano Cacciatore[1,2] stefano.cacciatore@icgeb.org
[1] *Bioinformatics Unit, International Center for Genetic Engineering and Biotechnology, Cape Town 7925, South Africa*
[2] *Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine (IDM), University of Cape Town, Cape Town 7925, South Africa*
[3] *Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy*
[4] *Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*

Editor: To be assigned

Abstract

The most widely used accelerated implementations of t-distributed stochastic neighbor embedding (t-SNE) and Uniform Manifold Approximation and Projection (UMAP) are fragmented across R, Python, C++, and accelerator-specific libraries. We present `fastEmbedR`, an R package that implements complete UMAP and openTSNE-style t-SNE workflows with native CPU, Apple Metal, and NVIDIA CUDA execution. The package accepts either a data matrix or reusable nearest-neighbor indices and distances, constructs UMAP graphs or t-SNE affinities internally, and executes optimization without calling Python. Its numerical core uses compact float32 and int32 storage, while GPU paths retain major working arrays on device. Backend-native randomized PCA provides t-SNE initialization, and explicit landmark functions support reference embedding and projection of new observations. In three-seed HPC benchmarks across 11 image, cytometry, metabolomics, and single-cell datasets, nine datasets supported matched full-pipeline comparisons. Median speedups were 1.81-fold for four-thread CPU and 54.2-fold for CUDA openTSNE relative to Rtsne; fuzzy UMAP speedups were 1.31-fold for CPU and 63.5-fold for CUDA relative to uwot fast SGD. Median trustworthiness differences were +0.039, +0.041, −0.0012, and −0.0012, respectively. `fastEmbedR` therefore makes accelerated embeddings directly available to R users through one coherent, inspectable API without concealing runtime or quality trade-offs.

Keywords: dimensionality reduction, t-SNE, UMAP, R software, GPU computing, Metal, CUDA

1 Introduction

t-SNE and UMAP are standard tools for high-dimensional biological and machine-learning data (van der Maaten and Hinton, 2008; McInnes et al., 2018; Kobak and Berens, 2019). Barnes–Hut t-SNE, FIt-SNE, openTSNE, and GPU UMAP improved their scalability

(van der Maaten, 2014; Linderman et al., 2019; Poličar et al., 2024; Nolet et al., 2020), but R users still encounter separate package interfaces, external executables, Python environments, and backend-specific data transfers. Moreover, a complete run includes neighbor search, affinity or graph construction, initialization, and iterative optimization; accelerating only one stage may not improve the user-visible call.

`fastEmbedR` addresses this software gap with one R interface backed by native CPU, Apple Metal, and NVIDIA CUDA implementations. It is not a wrapper around Python openTSNE or RAPIDS cuML. The package owns KNN-to-affinity/graph conversion, initialization, sparse schedules, optimization state, landmark projection, return objects, timing metadata, and backend validation. Optional platform libraries provide narrowly defined primitives such as CUDA search, truncated SVD, or FFTs. The resulting public API supports both complete matrix-input workflows and direct reuse of precomputed KNN matrices.

2 Software Architecture and Implementations

2.1 R interface and computational boundary

The matrix-input functions `opentsne()` and `umap()` accept ordinary R matrices. They also accept float32 matrices from the `float` package. The KNN-entry functions, `opentsne_knn()` and `umap_knn()`, accept reusable integer neighbor indices and floating-point distances. Backend selection is explicit: `backend="cpu"`, `"metal"`, or `"cuda"`. An unavailable GPU request fails rather than silently running on CPU.

For matrix input, CPU neighbor search uses a package-resident HNSW implementation distilled from permissively licensed FAISS code (Johnson et al., 2021; Douze et al., 2024); Metal uses native exact or recall-tuned IVF-Flat kernels; CUDA links to installed FAISS GPU exact search or the cuVS IVF-Flat C API and can keep KNN buffers on the device. The source distributions of these libraries are not vendored. The exported `pca()` function uses randomized/truncated SVD (Halko et al., 2011): CPU follows the `fastPLS` RSVD design, Metal uses a native float32 block-subspace method with MPS products, and CUDA uses RAFT TSVD. Setting `opentsne_init=TRUE` returns the small-scale PCA initialization used by modern t-SNE practice (Kobak and Berens, 2019; Cacciatore, 2026).

The portable core is compiled as C++17 and adds only POSIX-thread linkage to the flags selected by the active R installation. Metal builds link the Apple Metal/MPS frameworks; CUDA builds use C++17 NVCC translation units with explicit deployment architectures and an R-ABI-compatible host compiler. The package does not impose global `-march=native`, `-ffast-math`, or `-O3`; exact resolved flags and dependency architectures are recorded with each reproducibility bundle.

2.2 Native UMAP

Fuzzy UMAP estimates per-row ρ_i and σ_i, forms directed memberships, and combines reciprocal edges as $w_{ij} + w_{ji} - w_{ij}w_{ji}$ (McInnes et al., 2018). The explicit `graph_mode="binary"` alternative uses the symmetric KNN union with unit weights and is reported as an approximation. Both modes use the package's low-dimensional attraction/repulsion, negative sampling, learning-rate decay, and size-aware epoch policy.

The graph is stored once as CSR int32 offsets/indices plus float32 weights and edge schedules. CPU code parallelizes row-wise bandwidth estimation, graph construction, sparse initialization products, and edge updates. Metal uploads the schedule once and keeps coordinates and optimizer state in unified GPU memory. CUDA can consume GPU-resident KNN buffers, prepare the graph on-device, and execute the package-native atomic optimizer. `uwot` is a behavioral benchmark, not a source or runtime dependency (Melville, 2026).

2.3 Native openTSNE-style t-SNE

`opentsne_knn()` obtains row-conditional Gaussian probabilities by perplexity-matched bandwidth search, then symmetrizes and normalizes sparse affinities. Its two phases implement early exaggeration and normal optimization with automatic learning rates, gains, momentum, clipping, recentering, sparse attractive forces, and exact or interpolation-grid negative gradients (van der Maaten and Hinton, 2008; Linderman et al., 2019; Poličar et al., 2024; Belkina et al., 2019). No openTSNE Python/Cython code is called.

CPU execution reuses FFT plans, interpolation grids, and worker scratch. Metal performs grid scatter, package-native two-dimensional FFT convolution, interpolation, sparse attraction, and updates in Metal kernels; its validated 512×512 Stockham transform follows permissively licensed Apple GPU design ideas (Bergach, 2026). CUDA retains affinities, coordinates, gains, updates, cuFFT plans, and grids on-device and groups repeated iterations with CUDA Graph execution. This sparse-attraction/interpolated-repulsion organization follows FIt-SNE and t-SNE-CUDA without invoking their executables (Linderman et al., 2019; Chan et al., 2018).

2.4 Precision and scalable transforms

Working data, distances, weights, coordinates, gradients, gains, and workspaces use float32; graph indices use int32. Float32 R input avoids an initial double conversion and receives a float32 layout, although total process memory also includes R objects, indices, FFT grids, and library workspaces. `landmark_tsne()` and `landmark_umap()` embed a reference subset, interpolate remaining observations, apply a local affine correction, and optionally refine projected points. The t-SNE transform holds reference coordinates fixed as in Poličar et al. (2021); the result records the approximation and each timing component. Detailed equations and dependency ownership are in the supplement.

3 Evaluation

We evaluated 11 datasets spanning 873 to 5,220,347 observations and 11 to 16,384 variables. Each method was run with seeds 4, 17, and 42; $k = 30$ or perplexity 30; four requested CPU threads or one CUDA device; and its documented production optimizer. Competing methods on a machine used the same R environment, compiler toolchain, and thread controls. Cross-package comparisons report total public-function time because reference implementations expose different boundaries; KNN and initialization are not subtracted. Labels are used only for post hoc metrics (Venna and Kaski, 2001; Rousseeuw, 1987). Figures 1 and 2 show representative results; complete timings, memory, quality, parameters, failures, compiler configuration, and Python/RAPIDS comparisons are supplied in the supplement.

On MNIST70k, median total times were 69.56 and 1.94 seconds for CPU/CUDA openTSNE versus 125.83 seconds for Rtsne, and 50.04 and 0.99 seconds for CPU/CUDA fuzzy UMAP versus 65.34 seconds for uwot fast SGD. Across nine matched datasets, median speedups were 1.81 and 54.2 for CPU/CUDA openTSNE and 1.31 and 63.5 for CPU/CUDA fuzzy UMAP. Corresponding median trustworthiness differences were +0.039, +0.041, −0.0012, and −0.0012; median Preserve@30 differences for fuzzy UMAP were −0.0013 and −0.0024. CPU peak-RSS ratios were 0.81 versus Rtsne and 0.63 versus uwot. Matched claims exclude unavailable or failed rows.

Correctness was evaluated at the mathematical boundaries. On a 2,000-point MNIST validation, t-SNE affinity edge Jaccard was 1.000, weight Pearson correlation was 1.000, and weight Spearman correlation was 0.852 against Python openTSNE. The fuzzy UMAP graph shared all validated edges with `uwot::similarity_graph`; common-edge weight Pearson and Spearman correlations were 0.99999 and 0.99998. Across three MNIST seeds, openTSNE CPU/CUDA Procrustes correlations exceeded 0.9993 and neighbor stability@15 ranged from 0.929 to 0.953. These tests support behavioral equivalence without claiming bitwise identity.

4 Discussion and Availability

`fastEmbedR` exposes established t-SNE and UMAP objectives consistently across multicore CPU, Metal, and CUDA, with matrix/KNN inputs, float32 storage, backend-native PCA, and reference transforms. FAISS, cuVS, RAFT, cuFFT, and MPS provide optional primitives; affinity/graph construction and embedding optimization remain package code. A complete analysis stays in R: one-call functions include search and initialization, while KNN-entry functions expose a controlled reusable boundary and both routes share the same builders and optimizers.

Correctness is assessed behaviorally because stochastic embeddings need not have identical coordinates. Fixed seeds control sampling and initialization, although atomic GPU summation order can vary. Landmark functions embed an explicit reference subset, initialize remaining points by interpolation and local affine correction, and optionally refine them against the fixed reference; component timings keep the approximation visible.

Speed depends on hardware, dimension, KNN recall, connectivity, and initialization. Binary UMAP is an explicit approximation and fuzzy mode the standard comparison. Landmarking is not uniformly faster: on MNIST CPU, 50% landmark binary UMAP was 1.10-fold faster with trustworthiness changing by −0.0010, whereas landmark openTSNE was slower; failed CUDA landmark rows support no speed claim. `fastEmbedR` is MIT licensed at `https://github.com/tkcaccia/fastEmbedR`; the repository provides installation guidance, tests, benchmarks, and generated benchmark tables. A clean release commit and archival DOI will be frozen before submission.

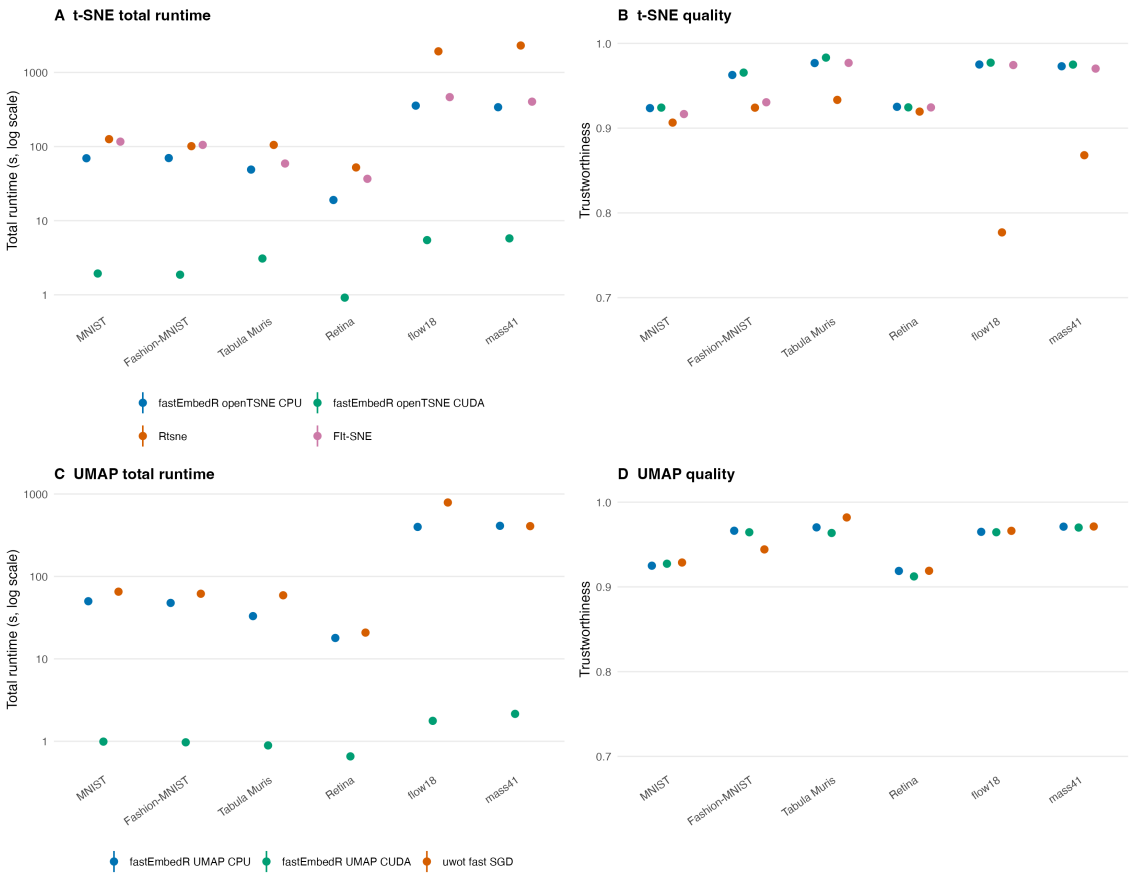

Figure 1: Runtime and quality on six representative HPC datasets. Points are medians over seeds 4, 17, and 42; runtime bars span the interquartile range and use a logarithmic axis. CPU methods requested four threads. Labels were used only for post hoc metrics.

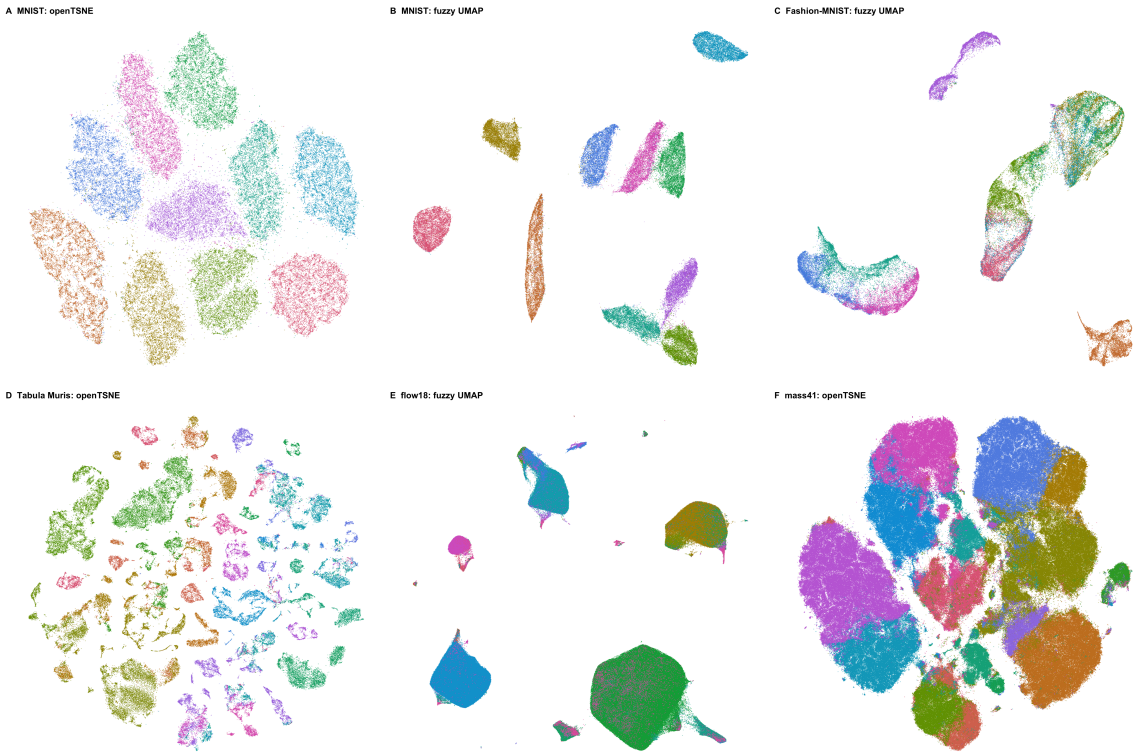

Figure 2: Representative label-colored CUDA embeddings from the archived HPC benchmark (seed 4). Labels were not supplied during fitting. Panels illustrate image, single-cell transcriptomic, flow-cytometry, and mass-cytometry inputs; plotting uses points only, without axes or boxes.

References

Anna C. Belkina, Christopher O. Ciccolella, Rina Anno, Richard Halpert, Josef Spidlen, and Jennifer E. Snyder-Cappione. Automated optimized parameters for t-distributed stochastic neighbor embedding improve visualization and analysis of large datasets. *Nature Communications*, 10:5415, 2019. doi: 10.1038/s41467-019-13055-y.

Mohamed Amine Bergach. AppleSiliconFFT. Software repository, 2026. URL `https://github.com/aminems/AppleSiliconFFT`.

Stefano Cacciatore. fastPLS: Accelerated partial least squares and randomized matrix decompositions for r. R package, 2026. URL `https://github.com/tkcaccia/fastPLS`.

David M. Chan, Roshan Rao, Forrest Huang, and John F. Canny. t-SNE-CUDA: GPU-accelerated t-SNE and its applications to modern data. *arXiv preprint arXiv:1807.11824*, 2018. doi: 10.48550/arXiv.1807.11824.

Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. doi: 10.48550/arXiv.2401.08281.

Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. doi: 10.1137/090771806.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.

Dmitry Kobak and Philipp Berens. The art of using t-SNE for single-cell transcriptomics. *Nature Communications*, 10:5416, 2019. doi: 10.1038/s41467-019-13056-x.

George C. Linderman, Manas Rachh, Jeremy G. Hoskins, Stefan Steinerberger, and Yuval Kluger. Fast interpolation-based t-SNE for improved visualization of single-cell RNA-seq data. *Nature Methods*, 16:243–245, 2019. doi: 10.1038/s41592-018-0308-4.

Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. doi: 10.48550/arXiv.1802.03426.

James Melville. uwot: The uniform manifold approximation and projection method for dimensionality reduction. R package, 2026. URL `https://github.com/jlmelville/uwot`.

Corey J. Nolet, Victor Lafargue, Edward Raff, Thejaswi Nanditale, Tim Oates, John Zedlewski, and Joshua Patterson. Bringing UMAP closer to the speed of light with GPU acceleration. *arXiv preprint arXiv:2008.00325*, 2020. doi: 10.48550/arXiv.2008.00325.

Pavlin G. Poličar, Martin Stražar, and Blaž Zupan. Embedding to reference t-SNE space addresses batch effects in single-cell classification. *Machine Learning*, 110:1211–1231, 2021. doi: 10.1007/s10994-021-06043-1.

Pavlin G. Poličar, Martin Stražar, and Blaž Zupan. openTSNE: A modular python library for t-SNE dimensionality reduction and embedding. *Journal of Statistical Software*, 109 (3):1–30, 2024. doi: 10.18637/jss.v109.i03.

Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. doi: 10.1016/0377-0427(87)90125-7.

Laurens van der Maaten. Accelerating t-SNE using tree-based algorithms. *Journal of Machine Learning Research*, 15:3221–3245, 2014. URL `https://www.jmlr.org/papers/v15/vandermaaten14a.html`.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. URL `https://www.jmlr.org/papers/v9/vandermaaten08a.html`.

Jarkko Venna and Samuel Kaski. Neighborhood preservation in nonlinear projection methods: An experimental study. In *Artificial Neural Networks – ICANN 2001*, pages 485–491. Springer, 2001. doi: 10.1007/3-540-44668-0_68.